

Competitive Programming

Section 34: Probability and Expected Value

Competitive Programming Course

2026-08-12

Table of contents

1	What an expected value problem is asking	3
2	Exact rational answers modulo a prime	4
3	Pattern 1: linearity of expectation	5
3.1	Worked trace	5
4	Pattern 2: first-step equations	6
4.1	Worked trace: success probability $2/5$	6
5	Pattern 3: coupon collector as staged waiting	7
5.1	Worked trace: $n=3$	7
6	Precompute harmonic sums efficiently	9
6.1	Why the inverse recurrence works	9
7	Debugging first-step models	11
7.1	Example with a self-loop	11
7.2	Example without a self-loop	11
7.3	A practical checklist	12
8	C++ and Python implementation notes	13
9	Formative exercises	14
10	Exercises	15
11	Local workflow and checklist	16

1 What an expected value problem is asking

Expectation is the average outcome over many independent repetitions of an experiment. It is not the most likely outcome. In contest problems it often lets us avoid listing exponentially many outcomes.

The three reliable entry points are:

Statement shape	First model to try
total reward is a sum of event rewards	indicator variables and linearity
repeat until a success	first-step equation / geometric wait
collect new kinds one by one	sum the waiting time of each stage

Write the random variable first. A formula without a random variable is hard to check and easy to apply to the wrong event.

2 Exact rational answers modulo a prime

The local problems ask for an exact rational number modulo $\text{MOD} = 1,000,000,007$, not a decimal approximation. Represent

a / b

by $a * b^{(\text{MOD}-2)} \bmod \text{MOD}$. This is valid because MOD is prime and every denominator used by the local contracts is nonzero modulo MOD .

For example, $1/3$ is the residue 333333336 , because

$$3 * 333333336 = 1,000,000,008 = 1 \pmod{\text{MOD}}.$$

Never use `double` and never use integer division. If a future problem allows a denominator divisible by MOD , this representation is not defined by a Fermat inverse; do not silently reuse this technique.

3 Pattern 1: linearity of expectation

For random variables X and Y ,

$$E[X + Y] = E[X] + E[Y].$$

Independence is **not** required for this equation. It is only needed for some probability products later in the course.

If bonus i has value $v[i]$ and happens with probability $p[i]/q[i]$, define

$$X_i = v[i] \text{ if bonus } i \text{ happens, otherwise } 0.$$

Then $E[X_i] = v[i] * p[i] / q[i]$, and the total reward is $\sum X_i$. Sum the individual expectations instead of enumerating all 2^n event subsets.

3.1 Worked trace

For bonuses $(p, q, v) = (1, 2, 10), (1, 4, 8), (3, 4, 12)$:

$$\begin{aligned} E[X_1] &= 10 * 1/2 = 5 \\ E[X_2] &= 8 * 1/4 = 2 \\ E[X_3] &= 12 * 3/4 = 9 \\ E[\text{total}] &= 5 + 2 + 9 = 16. \end{aligned}$$

The local problem describes independent events, but notice that the calculation above did not multiply event probabilities. It would remain valid if the bonus events were dependent and their individual probabilities stayed the same.

4 Pattern 2: first-step equations

Suppose one attempt succeeds with probability p . Let E be the expected number of attempts until the first success. We always spend one attempt. After a failure, probability $1-p$, we are back in the same state:

$$\begin{aligned} E &= 1 + (1-p)E \\ pE &= 1 \\ E &= 1/p. \end{aligned}$$

For a local input probability p/q , this is q/p modulo MOD .

4.1 Worked trace: success probability $2/5$

$$\begin{aligned} E &= 1 + (3/5)E \\ (2/5)E &= 1 \\ E &= 5/2. \end{aligned}$$

Modulo MOD , print `5 * inverse(2) = 500000006`. The contract for problem B has $1 \leq p \leq q < \text{MOD}$; success probability zero is intentionally excluded, because then no finite expectation exists. Treat that as a modeling warning, not as a hidden test case.

5 Pattern 3: coupon collector as staged waiting

There are n coupon types, and every draw independently returns each type with probability $1/n$. We want the expected number of draws needed to see all types.

Trying to describe the entire history is unnecessary. Split the process into stages:

```
stage i starts when exactly i distinct types have been collected;  
stage i ends when the next previously unseen type appears.
```

Let T_i be the number of draws spent in stage i . Every complete collection process passes through stages $0, 1, \dots, n-1$ exactly once, so its total number of draws is:

$$T = T_0 + T_1 + \dots + T_{(n-1)}.$$

Linearity of expectation then gives:

$$E[T] = E[T_0] + E[T_1] + \dots + E[T_{(n-1)}].$$

We can analyze one stage independently. When i distinct types are already collected:

- i types are old and keep us in the same stage;
- $n-i$ types are new and end the stage.

Because all types are equally likely,

$$P(\text{next draw is new}) = (n-i)/n.$$

Thus stage i is a repeated experiment whose success probability is $p=(n-i)/n$. Using the first-step result from the previous section, the expected number of draws until that success is:

$$E[\text{stage } i] = n/(n-i).$$

This quantity includes the successful draw that discovers the next type. For example, with $n=3$, after collecting one type, each draw has probability $2/3$ of being new. We may succeed immediately, or draw the old type several times first; the average stage length is $1/(2/3)=3/2$ draws.

Adding the n stage expectations gives:

$$\begin{aligned} & n/n + n/(n-1) + \dots + n/1 \\ &= n * (1 + 1/2 + \dots + 1/n). \end{aligned}$$

5.1 Worked trace: $n=3$

collected i	chance of new type	expected draws for this stage
0	$3/3$	1
1	$2/3$	$3/2$
2	$1/3$	3
total		$11/2$

Modulo MOD, $11/2$ is 500000009. The stage expectation is valid even though the actual stage lengths are random and need not be independent. Linearity of expectation applies to their sum regardless of dependence.

6 Precompute harmonic sums efficiently

For up to 200,000 coupon queries with $n \leq 1,000,000$, compute

```
H[i] = H[i-1] + inverse(i),    H[0] = 0.  
answer(n) = n * H[n].
```

Calling binary exponentiation for every inverse costs $O(N \log \text{MOD})$ and can be acceptable in optimized C++, but a linear inverse table is the better reusable tool:

```
inv[1] = 1  
inv[i] = MOD - (MOD / i) * inv[MOD % i] mod MOD, for i >= 2.
```

6.1 Why the inverse recurrence works

Fix an i with $2 \leq i < \text{MOD}$. Integer division of MOD by i gives:

```
MOD = floor(MOD/i) * i + (MOD mod i).
```

Use shorter names:

```
q = floor(MOD/i)  
r = MOD mod i
```

Then:

```
MOD = q*i + r.
```

We now work modulo MOD . Since MOD itself is congruent to zero:

```
q*i + r == 0 (mod MOD)  
r == -q*i (mod MOD).
```

Because MOD is prime and $1 \leq i < \text{MOD}$, the remainder satisfies $1 \leq r < i$; in particular, it is nonzero and has an inverse. Multiply the last congruence by $\text{inv}[r]$:

```
r * inv[r] == -q*i*inv[r] (mod MOD).
```

By definition, $r \cdot \text{inv}[r] == 1$, so:

```
i * (-q*inv[r]) == 1 (mod MOD).
```

Therefore $-q \cdot \text{inv}[r]$ is an inverse of i . Normalize this negative expression into the usual modular range:

```
inv[i] = MOD - q*inv[r] mod MOD
        = MOD - floor(MOD/i)*inv[MOD mod i] mod MOD.
```

The recurrence is usable from left to right because $r = \text{MOD} \bmod i$ is strictly smaller than i . Thus, when computing $\text{inv}[i]$, $\text{inv}[r]$ has already been computed.

For a small trace, take the prime modulus 17 and compute the inverse of 5:

```
17 = 3*5 + 2, so q=3 and r=2.
inv[2] = 9 because 2*9 == 1 (mod 17).

inv[5] = -3*inv[2] mod 17
        = -27 mod 17
        = 7.
```

Check:

```
5*7 = 35 == 1 (mod 17).
```

In code, write the normalization explicitly:

```
inv[i] = MOD - (MOD / i) * inv[MOD % i] % MOD;
```

This proof requires every i being inverted to be nonzero and invertible modulo MOD . The local prime modulus and $N < \text{MOD}$ guarantee that. With a composite modulus, $\text{MOD} \% i$ may not be invertible and this table recurrence cannot be used blindly.

The table gives $O(N)$ preprocessing and $O(1)$ per query.

7 Debugging first-step models

First-step analysis means: condition on what happens during the very next action, then express the remaining work using expectation states.

For an expectation state $E[\text{state}]$, write the equation in three pieces:

```
expected total work
= work paid immediately
+ expected work remaining after the next transition.
```

Equivalently:

```
E[state] = immediate cost
          + sum over outcomes:
              P(outcome) * E[state after outcome].
```

The probabilities in that sum must cover every possible next outcome and add to one.

7.1 Example with a self-loop

An attempt costs one step. It succeeds with probability p and stops; it fails with probability $1-p$ and returns to the same state. Therefore:

```
E = 1 + p*0 + (1-p)*E.
```

The E on the right is not circular reasoning. It says that after a failure, the process is in exactly the same situation as before, so the expected remaining work is again E . Solve the equation algebraically:

```
E - (1-p)E = 1
pE = 1
E = 1/p.
```

Do not implement this as a recursive call to the same state: it would recurse forever. The self-loop term must first be moved to the left and eliminated algebraically.

7.2 Example without a self-loop

Suppose a state has two equally likely outcomes: move to state A or state B, paying one step now. Then:

$$E[\text{state}] = 1 + (1/2)E[A] + (1/2)E[B].$$

If $E[A]$ and $E[B]$ are already known, this is an ordinary DP transition. More generally:

- if every transition goes to an already computed state, evaluate states in dependency order;
- if a transition returns to the same state, isolate that state algebraically;
- if several expectation states depend cyclically on one another, the model produces a system of linear equations rather than a simple DP.

7.3 A practical checklist

Before trusting a first-step equation, verify:

What exactly does $E[\text{state}]$ measure?

What cost is paid before the next state is reached?

Have I listed every possible next outcome exactly once?

Do their probabilities sum to 1?

Which outcomes stop and therefore contribute future expectation 0?

Which outcomes return to the current state?

Most mistakes come from omitting the immediate +1, forgetting a self-loop, or treating a transition probability as an expected number of steps.

8 C++ and Python implementation notes

In C++, keep every modular product in `long long`; reduce after each product. For Coupon Collector, allocate `vector<long long>` once through the maximum query and use fast buffered I/O. In Python, use `pow(x, MOD-2, MOD)` for a single inverse and keep harmonic/inverse tables in lists; read all queries first to know the required maximum. The linear inverse recurrence reduces Coupon Collector preprocessing from $O(N \log \text{MOD})$ to $O(N)$.

9 Formative exercises

1. A reward of 9 is gained with probability $2/3$, and a reward of 6 is gained with probability $1/2$. What is the expected total? Would your answer change if the two events were dependent?
2. An attempt succeeds with probability $1/4$. Write, then solve, the first-step equation for its expected number of attempts.
3. For three coupon types, what is the probability of a new type after exactly one distinct type has been collected? What is that stage's expectation?
4. Why is $p=0$ not a valid input for First Success, while $p=0$ is valid for an individual Expected Prize event?

Answers:

1. $9 \cdot (2/3) + 6 \cdot (1/2) = 9$. No: linearity does not require independence.
2. $E = 1 + (3/4)E$, so $E = 4$.
3. $2/3$, then $3/2$.
4. A zero-probability prize simply contributes expectation zero. Repeating an experiment that can never succeed has infinite expected stopping time.

10 Exercises

Complete the four formative exercises above before local Problems A–C. They move from linearity, to a self-loop equation, to staged expectation.

11 Local workflow and checklist

Run:

```
CP_TARGET=solution python3 sections/34_probability_expected_value/check.py
```

Before submission, check:

```
I defined a random variable or expectation state.  
I used linearity without adding an unnecessary independence assumption.  
I multiplied probabilities only where independence is stated.  
Every modular denominator is nonzero modulo MOD.  
The first-step equation includes the attempt made now.  
Coupon stages run from i=0 through n-1, not in the reverse direction.
```